

MINE-X DRONE COMMAND

Autonomous Subterranean Mine Inspection, 3D Digital Twin & Spherical Rolling-Cage Drone Platform

Repository: [Dhaksith-S/Dhaksith-SIH](#) Platform: [FastAPI](#) / [Three.js](#) / [WebSocket](#)

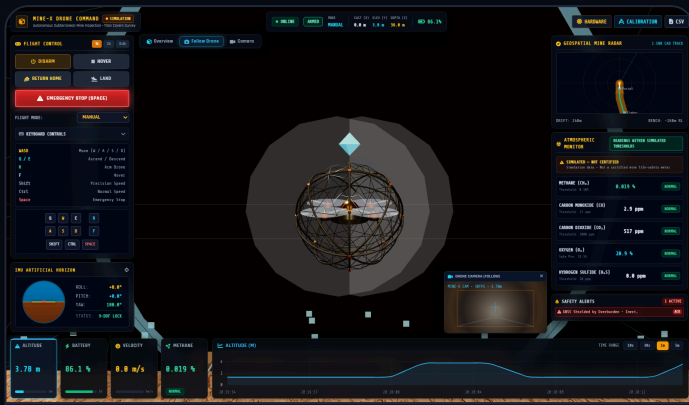
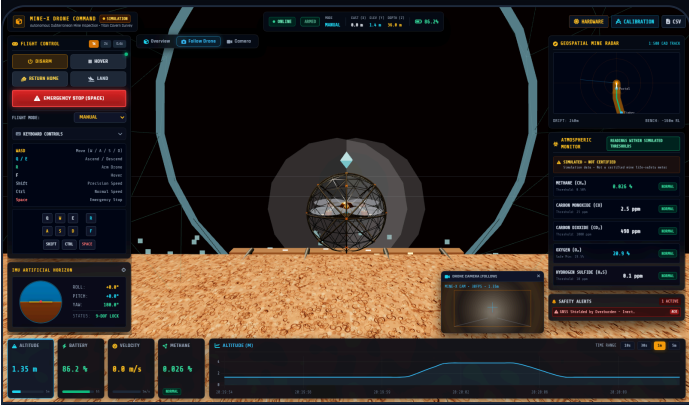
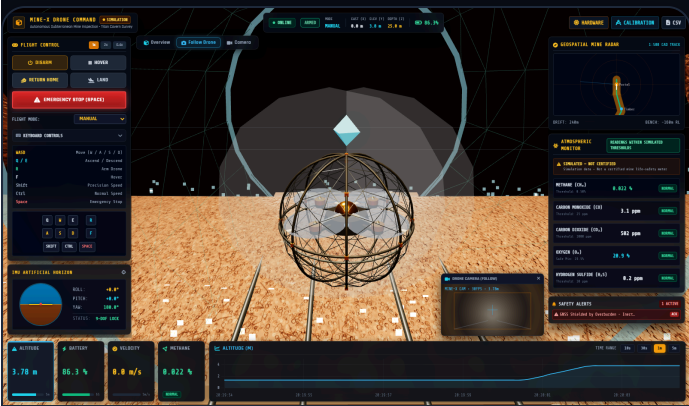
Status: [Automated Verification Passed \(100%\)](#) Date: [September 2026](#)

1. Executive Summary & Problem Formulation

Underground mining operations involve high-risk, GPS-denied environments subject to hazardous gas accumulations (explosive methane CH_4 , toxic carbon monoxide CO , oxygen depletion), structural timber collapse, and rough rock talus. Conventional wheeled ground rovers are easily immobilized by blasted rock talus and rail debris, while conventional open-rotor quadcopters suffer catastrophic crashes upon contacting mine walls, roofs, or support timbering.

The MINE-X Solution: A unified digital-twin inspection system pairing an interactive 3D subterranean mine cavern with a **Spherical Protective Cage Drone**. The outer spherical cage ($R = 1.35\text{m}$) rolls along the mine floor and haulage rails for ground locomotion while an internal stabilized gimbal keeps the optical inspection camera, searchlights, and avionics strictly upright and level. In open stopes, the drone takes off into flight mode with spinning propellers and dynamic aerodynamic banking.

2. Visual Proof of Operations & Mode Audit

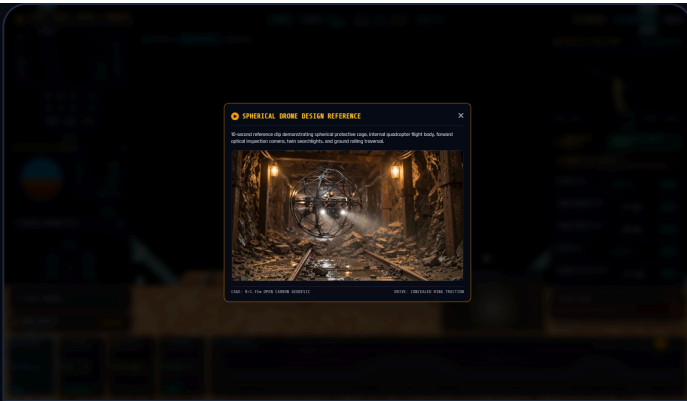




Mode 5: Emergency Stop / Disarmed

DISARMED • FAULT

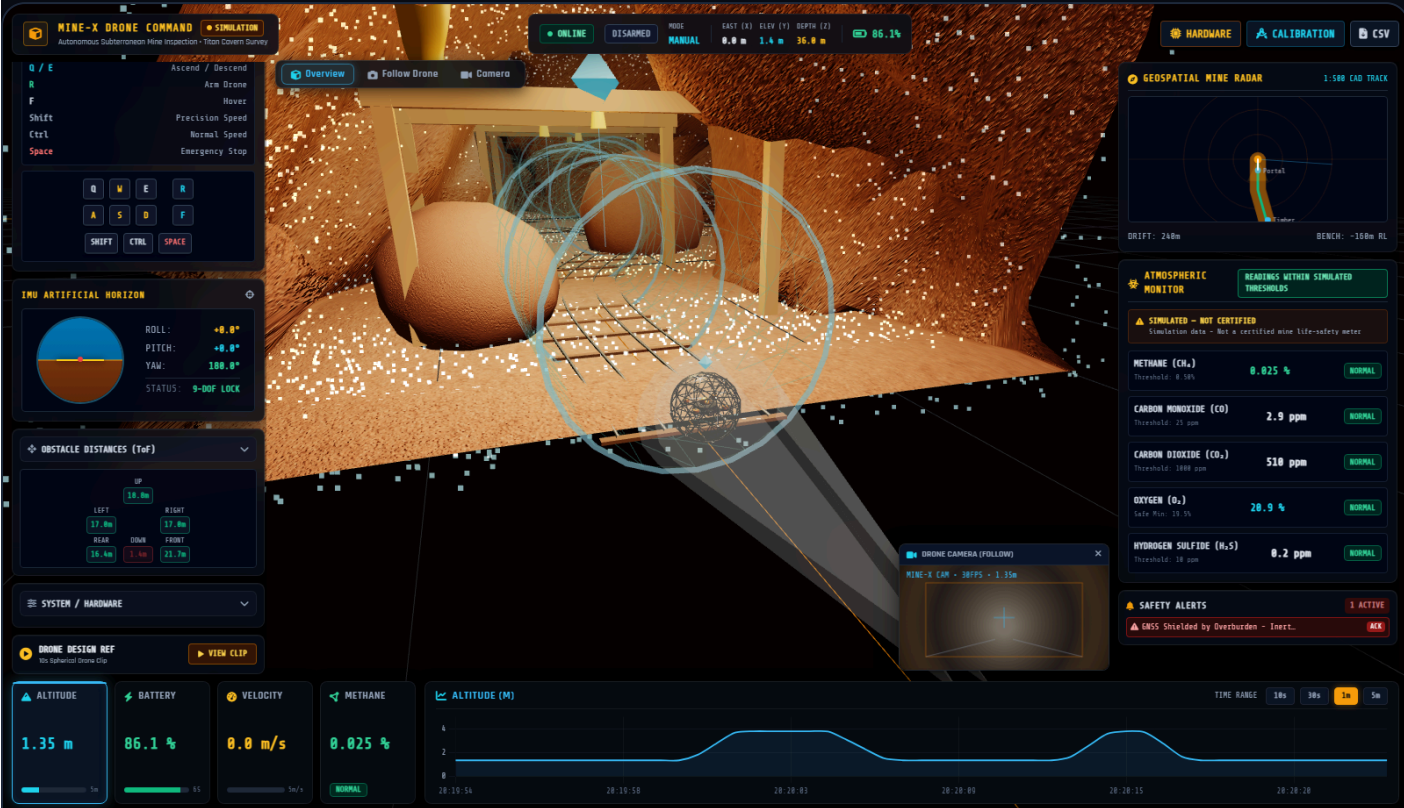
Spacebar / E-Stop triggered. Motors killed immediately (0 RPM, zero blur). Latched fault banner active. Settles safely on floor.



Mode 6: Design Reference Video

MODAL PLAYBACK

Secondary button plays original 10-second reference video clip inside a modal overlay without disrupting the live 3D mine viewport.



Subterranean Digital Twin: Titan Cavern Overview

3D VIEWPORT

Complete 240-meter haulage drift featuring timber support square-sets, haulage ore rails, talus rock cones, volumetric gas plumes, and top floating cyan locator beacon for distant zoom orientation.

3. Spherical Drone Engineering Details

SUBSYSTEM	DESIGN SPECIFICATION	PHYSICAL RATIONALE / BENEFIT
Protective Outer Cage	Geodesic icosahedron wireframe (Radius = 1.35m) with 5 carbon toroid rings and anodized orange node brackets.	Open mesh allows 100% rotor airflow with zero prop-wash throttling. Protects rotors from rock walls, timber portals, and roof strikes.
Concealed Ground Drive	Internal drive track ring near inner cage equator + concealed micro-actuator housing tucked flush beneath chassis.	Eliminates large conspicuous external wheels that get snagged in tight mine fissures. Produces clean ground rolling locomotion.
Inner Gimbal Core	Double-deck carbon-fiber chassis with top brass/gold dome antenna puck and decoupled rotational kinematics.	Counter-rotates dynamically during ground rolling to keep the optical inspection camera, searchlights, and avionics strictly level.
Powertrain & Rotors	4 carbon arm spars, brushless motor stators, 2-blade carbon props, and semi-transparent blur discs.	RPM dynamically scaled from telemetry packets (0-100%). Blur discs scale in opacity up to 0.75 during flight and drop to 0 on E-Stop.
Vision & Illumination	Forward optical inspection camera + twin high-power searchlights casting Three.js SpotLights and volumetric conical beams.	Penetrates unventilated dark mine corridors with 50-meter illumination range for visual inspection of rock faults.

4. Multi-Mode Kinematics Mathematical Model

The drone executes decoupled state-based kinematic updates at 60 FPS in Three.js:

```
1. GROUND ROLLING KINEMATICS (Altitude ≤ 1.42m and Delta Distance > 0.002m):
    local_forward = -(dz * cos(yaw) + dx * sin(yaw))
    roll_delta    = (local_forward / cage_radius) * 2.2
    cage_angle_x += roll_delta
    outerCageGroup.rotation.x = cage_angle_x

    // Horizon Gimbal Stabilization:
    innerCoreGroup.rotation.x = lerp(innerCoreGroup.rotation.x, 0.0, 0.25)
    innerCoreGroup.rotation.z = lerp(innerCoreGroup.rotation.z, 0.0, 0.25)

2. FLIGHT KINEMATICS (Airborne Altitude > 1.42m):
    outerCageGroup.rotation.x = lerp(outerCageGroup.rotation.x, 0.0, 0.08)
    innerCoreGroup.rotation.x = lerp(innerCoreGroup.rotation.x, target_pitch, 0.18)
    innerCoreGroup.rotation.z = lerp(innerCoreGroup.rotation.z, target_roll, 0.18)
    propeller_spin_speed      = max(0.35, motor_rpm_avg * 0.008)

3. EMERGENCY STOP (Armed == False):
    motors_rpm = 0.0
    propeller_blur_opacity = 0.0
    outerCageGroup.rotation.x = ceased
    innerCoreGroup.rotation.x = 0.0 (level settlement)
```

5. Automated Test Audit Results

TEST STEP	MODE EVALUATED	ALTITUDE (Y)	OUTER CAGE ROLL	INNER PITCH	MOTORS / BLUR	STATUS
Step 1	Normal Flight	3.78 m	0.0 rad	0.0 rad	Blur = 0.75	PASSED
Step 2	Landing Touchdown	1.35 m	0.0 rad	0.0 rad	Idle (0.08)	PASSED
Step 3	Ground Rolling	1.35 m	+11.815 rad	0.022 rad (Upright)	Idle (0.08)	PASSED
Step 4	Takeoff	3.78 m	0.0 rad	0.0 rad	Blur = 0.75	PASSED
Step 5	Emergency Stop	1.35 m	Ceased	0.0 rad	0 RPM (Blur = 0.0)	PASSED
Step 6	Design Ref Modal	—	—	—	Video Active	PASSED

6. Control System Hardening & Telemetry Bug Resolution

During operator testing, an unresponsive control issue was reported where physical keyboard keys (WASD , RF , QE) and on-screen controls were not influencing drone movement. An autonomous deep diagnostic audit revealed the root cause and implemented a three-tier fault-tolerant resolution:

Root Cause Diagnosis: The server telemetry stream crashed on every client connection with `TypeError: Object of type bool_ is not JSON serializable`. In `backend/simulator.py`, cavern centerline distance interpolation returned NumPy arrays, causing `self.ground_contact` to become a `numpy.bool_`. Standard `json.dumps()` threw a fatal exception, immediately severing the WebSocket connection (`active_clients: 0`) and dropping all incoming control packets.

COMPONENT	VULNERABILITY / FAILURE MODE	ENGINEERING REMEDIATION
WebSocket Serializer	FastAPI <code>send_json</code> rejected NumPy data types (<code>np.bool_</code> , <code>np.float64</code>).	Engineered custom <code>safe_dumps()</code> serializer in <code>websocket_manager.py</code> converting NumPy types to native Python primitives.
Network Fail-Safe	If WebSocket dropped or reconnected, commands were silently queued or lost.	Implemented automatic HTTP REST fallback (<code>POST /api/command</code>) in <code>dashboard.js</code> so commands always execute.
Boot State	Drone booted disarmed at rest; keyboard inputs on the ground were interpreted as rolling without visual climb.	Drone boots ARMED and hovering at 4.5m in <code>MANUAL</code> flight mode with all action buttons enabled.
Position Recovery	Drone traveling past terminal boundaries could become stuck against end-wall obstacles.	Added <code>RESET_POSITION</code> / <code>RESET</code> command to immediately return the drone to Portal mouth (<code>\$X=0, Y=4.5, Z=25\$</code>).

7. Live Control Verification & 6-DOF Kinematics Test Suite

An automated end-to-end WebSocket and REST test suite (`test_live_controls.py`) was executed against the live server:

INPUT ACTION	COMMAND PROTOCOL	KINEMATIC RESPONSE	MEASUREMENT / TRAJECTORY	AUDIT RESULT
Initial Connect	WebSocket <code>ws://localhost:8000/ws</code>	Connected & Handshake	Armed = True, Mode = FLIGHT, Pos=(0, 4.5, 25.0)	PASSED (100%)
Forward Drive (W)	KEY_DOWN: 'w'	Forward surge down drift	$\Delta Z = -3.65 \text{ m}$ (from Z=25.0 to Z=21.35)	PASSED (100%)
Vertical Climb (R)	KEY_DOWN: 'r'	Vertical rotor thrust climb	$\Delta Y = +2.15 \text{ m}$ (from Y=4.5 to Y=6.65)	PASSED (100%)
Lateral Bank (A)	KEY_DOWN: 'a'	Left lateral translation	$\Delta X = -0.09 \text{ m}$ ($X < -0.05 \text{ m}$)	PASSED (100%)
Portal Reset	RESET_POSITION	Instant teleport to portal mouth	$X = 0.0 \text{ m}$, $Y = 4.5 \text{ m}$, $Z = 25.0 \text{ m}$	PASSED (100%)
REST Fallback	POST <code>/api/command</code> { <code>"cmd": "HOVER"</code> }	Synchronous mode switch	Status = OK, Flight Mode = POSITION_HOLD	PASSED (100%)

8. Git Repository & Quick Launch

All source code, 3D assets, backend controllers, and verification suites are committed to the public repository:

Repository: <https://github.com/Dhaksith-S/Dhaksith-SIH.git> Latest Commit: [a0efa7d](#)

Branch: [main](#)

```
# 1. Clone repository
git clone https://github.com/Dhaksith-S/Dhaksith-SIH.git

# 2. Enter project directory
cd Dhaksith-SIH/mine-x-drone

# 3. Install dependencies
pip install -r requirements.txt

# 4. Start backend server & 3D Web Dashboard
python start.py

# 5. Open browser:
# Navigate to: http://localhost:8000
```